

EXP 8: LINEAR REGRESSION

NAME: SAMUEL I

REG NO:192312584

AIM:

To implement the Linear Regression algorithm in Python and predict continuous values using a given dataset.

PROCEDURE

1. Import the required libraries and load the dataset into the program.
2. Split the dataset into training data and testing data.
3. Train the Linear Regression model using the training dataset.
4. Predict the output values for the testing dataset.
5. Display the Mean Squared Error and R^2 score to evaluate the performance of the model.

PROGRAM:

Linear Regression using Python

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
data = load_diabetes()
X = data.data
y = data.target
```

```
# Split dataset

X_train, X_test, y_train, y_test =
    train_test_split(X, y, test_size=0.3,
        random_state=42
    )

# Create and train model

model = LinearRegression()
model.fit(X_train, y_train)

# Predict test data

y_pred = model.predict(X_test)

# Display results

print("Predicted Values:")
print(y_pred)

print("\nMean Squared Error:")
print(mean_squared_error(y_test, y_pred))

print("\nR2 Score:")
print(r2_score(y_test, y_pred))
```

OUTPUT:

```
Predicted Values:
[138.46970031 181.10052342 125.34400904 292.75977277 123.8830531
 91.89830434 257.26463123 177.76169318 84.98549706 109.15960992
 94.4855284 166.13235108 57.40742502 206.13897354 97.7811842
131.00472765 214.29789972 252.52907661 199.66656381 219.49985634
203.23671317 88.00656925 70.65108459 186.92233427 155.81266751
162.81022205 191.93135706 184.72924276 46.62920829 108.26566599
178.14743952 91.35065005 127.72125745 184.04205666 172.23799897
189.51548863 121.83265708 117.75339206 145.67950306 58.57563401
72.55421321 107.27571105 166.15280313 153.29339984 165.19282154
62.95752038 73.58909449 110.05656189 54.49723354 164.88920632
152.49472803 63.8874565 111.4354561 108.34936269 179.96973311
158.70054112 95.04833555 209.68990406 118.37356519 69.69946093
189.04680627 204.99138626 140.26840176 105.75963198 131.22316354
201.65649349 173.60251683 164.25810664 122.01567481 143.53425629
179.42286037 201.32086534 234.50804093 146.96093984 80.27567403
160.86822173 192.72110552 208.6585054 158.56823462 206.00933474
111.53729194 140.28763033 51.15323818 54.76434639 113.02207773
78.46425825 80.20416459 56.97774667 168.51207071 185.55749267
152.33436369 242.66080008 106.44250735 61.34968205 53.44975218
197.78538801 246.62420102 185.25237675 105.80114548 62.42547543
193.39002547 107.08821595 297.08578592 94.45306615 148.41869832
104.10666888 128.66940497 126.90012169 165.84858396 186.14546255
108.5888133 203.83170771 204.97622269 209.10156998 191.65509488
161.26918179 148.51075431 135.0455994 111.74197539 163.62966339
170.55301782 95.53608319 46.85961143 164.34533761 127.1747026
176.61035076 173.47305611 107.40153597 202.99778708 92.10100436
122.28876806 80.80866991 233.21870492]
```

2.

PROGRAM:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
X, y = fetch_california_housing(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

OUTPUT:

```
MSE: 0.5558915986952422
R2 Score: 0.5757877060324524
```

3.

PROGRAM:

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
X, y = load_diabetes(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

OUTPUT:

```
MSE: 2900.193628493482
R2 Score: 0.4526027629719195
```

4.

PROGRAM:

```
from sklearn.datasets import make_friedman2
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
X, y = make_friedman2(
```

```

n_samples=1000,
noise=5.0,
random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

```

OUTPUT:

```

MSE: 19731.099214122518
R2 Score: 0.8803231885274723

```

5.

PROGRAM:

```

from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

```

```

X, y = make_regression(
    n_samples=1000,
    n_features=8,
    noise=15,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42

```

)

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("MSE:", mean_squared_error(y_test, y_pred))
```

```
print("R2 Score:", r2_score(y_test, y_pred))
```

OUTPUT:

```
MSE: 233.2442442099851  
R2 Score: 0.9921508554765083
```

RESULT:

The Logistic Regression algorithm was successfully implemented in Python. The model was trained and tested on the dataset, and its performance was evaluated using the confusion matrix and accuracy, achieving effective classification results.